

CAF Current State — Vol 1: Platform & Funnel

Aggregated export — 2026-07-10. Source markdown in repo is unchanged.

Bundle id: 11-caf-current-state-vol1-platform

Included:

docs/volumes/CAF_CONTEXT_VOL1_Platform_and_Funnel.md

CAF Current State Context Pack — Platform and Funnel

Volume 1 of 4 | Generated 2026-07-10 | Full pack: docs/CAF_CURRENT_STATE_CONTEXT_PACK.md

1. Executive summary

What CAF is

CAF (Content Automation Framework) is a **content automation / content operations platform** that turns research inputs, evidence, signal packs, creative intelligence, and planning decisions into **generated, QC-checked, rendered, reviewed, publishable, and learnable** social content.

The production loop:

```
Inputs / evidence → signal pack → planned jobs → decision engine → content jobs
  → LLM drafts → QC / risk → diagnostics → rendering → human review
  → rework → publishing → performance metrics → learning rules
```

Problem it solves

Operators run multi-platform content brands (e.g. SNS, Cuisina) at scale: ingest competitor/top-performer research, plan ideas per flow type, generate copy and media, enforce brand/risk gates, route through human review, publish to Meta, and close the loop with performance-driven learning.

Who uses it

Persona	Surface
Marketer / brand owner	Review app marketer funnel (/workspace, /brand/[slug]/*)
Content operator / reviewer	Review workbench (/review, /t/[task_id])
Pipeline engineer	Core API, CLI (npm run process-run), admin HTML
Inputs / research operator	Admin inputs pages, /v1/inputs-* APIs
External workers	n8n publish webhooks, renderer/video-assembly sidecars

Source of truth

- PostgreSQL schema `caf_core` — especially `content_jobs.generation_payload`
- Core API (`src/server.ts` , Fastify) — not the Review app
- Migrations (`migrations/*.sql`) — schema truth over prose docs

Mature vs experimental (honest)

Mature / production-used	Partial / gated / experimental
Run planning + job pipeline	Product image flows (FLOW_IMG_*) — blocked at generation
Standard carousel (FLOW_CAROUSEL)	Full composite saliency text-placement automation (designed, not built)
QC + risk policies + human review	Project risk_rules — config only, not QC-enforced
HeyGen video flows	Scene assembly (Sora) — wired but provider-dependent
Inputs → signal pack funnel	Stage-3 structured idea picker at plan time (partial)
Publications + Meta executor	CAF_PUBLISH_EXECUTOR=none default (n8n external)
Learning rules (planning + generation)	Global learning rules (disabled in compiler)
Review app workbench	Marketer funnel — growing, not all pipeline controls
Manual mimic carousel	Why Mimic + New Visual — newer, actively evolving
Brand Visual System (BVS) + brand bibles	BVS invented plates — <code>template_bg</code> + BVS only

2. Platform components

Component	Path	Purpose	State owned	Depends on	Key env
CAF Core API	<code>src/</code>	Orchestration, HTTP, business logic	<code>caf_core.*</code> tables	Postgres, OpenAI, providers	DATABASE_URL, PORT
Postgres <code>caf_core</code>	<code>migrations/</code>	All persistent state	Everything	—	DATABASE_URL
Review app	<code>apps/review/</code>	Operator + marketer UI (client)	None (proxies Core)	Core API, renderer	CAF_CORE_URL, RENDERER_BASE_URL
Admin HTML	<code>src/routes/admin.ts</code>	Legacy ops workbench	None	Core	Same as Core
Renderer	<code>services/renderer/</code>	Puppeteer + HBS → PNG slides	Temp files	Templates from Core or disk	PORT, RENDER_CONCURRE
Video assembly	<code>services/video-assembly/</code>	ffmpeg stitch/mux/burn	Temp uploads	Supabase	SUPABASE_*
Media gateway	<code>services/media-gateway/</code>	Single port proxy to	None	Child processes	PORT, SPAWN_CHILDREN

		renderer + video			
Storage	Supabase	Asset URLs (carousel, video, mimic plates)	Bucket objects	—	SUPABASE_URL, SUPABASE_SERVICE_ROLE
OpenAI	external	LLM generation, TTS, Sora, vision	None	API key	OPENAI_API_KEY
HeyGen	external	Avatar / video agent renders	Provider session IDs in render_state	API key	HEYGEN_API_KEY
BFL / DashScope / NVIDIA	external	Mimic image edit / T2I	None	Provider keys	MIMIC_IMAGE_*, BFL_API_KEY
Meta Graph	external	IG/FB publish	Placement rows	Page tokens	CAF_PUBLISH_EXECUTOR, CAF_META_*
Apify / scrapers	external	Evidence ingest	Scraper run rows	APIFY_API_TOKEN	Partial
Document AI	external	Carousel OCR for mimic overlay seed	None	GCP creds	DOCUMENT_AI_*
n8n / external publish workers	external	Legacy publish execution	Callback to Core	Webhook	CAF_PUBLISH_EXECUTOR

Deployment: Production Review is embedded at `https://caf-core.fly.dev/admin/workbench` (not a separate Vercel deploy). Ship Review changes with `fly deploy -a caf-core` from repo root.

3. End-to-end funnel

Stage map

Stage	Trigger	Main files	DB tables	Input → output
Evidence upload	POST /v1/inputs-evidence/upload	inputs-evidence.ts, repos	inputs_evidence_imports, inputs_evidence_rows	XLSX → normalized rows
Scraper runs	POST /v1/inputs-sources/...	inputs-scrappers.ts	inputs_scraper_runs, inputs_source_rows	Apify/HTML → evidence shape
Insights passes	POST /v1/inputs-processing/.../run-*	inputs-processing.ts	inputs_evidence_row_insights	Rows → tier insights

Signal pack build	POST .../build-signal-pack	processing + signal-packs	signal_packs, ideas	Import/idea list → pack JSON
Run create	POST /v1/runs	runs.ts, run-orchestrator.ts	runs	project + signal_pack_id
Materialize jobs	POST .../jobs OR .../candidates	runs.ts, run-candidates-materialize.ts	runs.planned_jobs_json	pack → planned rows
Start run	POST .../start	run-orchestrator.ts	content_jobs, runs	planned → jobs
LLM generation	pipeline auto / POST .../generate	llm-generator.ts, job-pipeline.ts	job_drafts, generation_payload	prompt → generated_output
Mimic prep	pipeline	mimic-draft-prep.ts, new-visual-carousel-prep.ts	generation_payload.mimic_v1	references + mode
QC	pipeline auto	qc-runtime.ts	generation_payload.qc_result	output → checklist + risk
Render	pipeline auto	job-pipeline.ts, render services	assets, render_state	payload → media URLs
Human review	operator	Review app → Core /decide	editorial_reviews	decision + overrides
Rework	NEEDS_EDIT	rework-orchestrator.ts	new draft attempt	feedback → regen
Publish	POST .../publications/.../start	publications.ts, meta-graph-publish.ts	publication_placements	approved job → platform post
Learning	cron / manual APIs	learning.ts, editorial-learning.ts	learning_rules, learning_observations	outcomes → rules

Run lifecycle (exact strings)

CREATED → PLANNING → PLANNED → GENERATING → (RENDERING) → (REVIEWING) → COMPLETED | FAILED | CANCELLED

Set in: migrations/002_project_config_and_runs.sql ,
src/repositories/runs.ts , run-orchestrator.ts , job-pipeline.ts , runs.ts .

Job lifecycle (exact strings — no DB CHECK on content_jobs.status)

PLANNED → GENERATING → GENERATED → (RENDERING) → IN_REVIEW | BLOCKED | REJECTED | NEEDS_EDIT | APPROVED | FAILED

Also observed: READY_FOR_REVIEW (legacy progress counting), QC_FAILED (only pipeline.ts /full endpoint, not main pipeline).

4. Domain model and IDs

Entities

Entity	Description	Primary key pattern
Project	Named content brand	project_id (uuid), project_slug (text, e.g. SNS)
Run	One execution cycle	run_id = {PROJECT}_{period} e.g. SNS_2026W09
Signal pack	Research bundle for a run	signal_pack_id (uuid/text per schema)
Candidate / planned row	Idea × flow in memory or planned_jobs_json	candidate_id = {run_id}_{platform}_{NNNN} or {base}_{flow_type}
Content job	Atomic executable unit	task_id = {run_id}_{platform}_{flow_type}_row{NNNN}_{variation}
Job draft	One LLM attempt	draft_id = d_{random12}
Asset	Rendered media artifact	asset_id = {candidate_id}_{ASSET_TYPE}_v{version}
Editorial review	Human decision	(project_id, task_id, review_id)
Publication placement	Scheduled post	uuid row in publication_placements
Learning rule	Structured behavior change	uuid in learning_rules
Evidence import	XLSX/scrapper batch	inputs_evidence_imports.id
Evidence row	Single post/account row	per import
Evidence insight	Tier analysis on a row	inputs_evidence_row_insights
Creative intelligence asset	Archived top-performer media	creative_intelligence_assets
Brand bible	BVS source per project	brand_bibles versioned rows
Brand profile	Marketer voice/strategy	brand_profiles

Join pattern

All job-related tables join on (project_id, task_id) — text IDs, not UUID FKs to content_jobs .

Legacy / dual-write

Legacy	Canonical	Helper
<code>runs.candidates_json</code>	<code>runs.planned_jobs_json</code>	<code>readRunPlannedJobsJson()</code> in <code>jobs-json-compat.ts</code>
<code>signal_packs.ideas_json</code>	structured idea tables	both may coexist
Old flow type names	<code>canonical-flow-types.ts</code>	<code>LEGACY_FLOW_TYPE_TO_CANONICAL</code>

Included: AGENTS.md

AGENTS.md — Guidance for AI coding assistants

This file helps **Cursor agents** (and humans) work safely and efficiently in **CAF Core**.

Project identity

- **Repo:** CAF Core — Fastify + PostgreSQL backend for a **content automation pipeline** (signals → jobs → LLM → QC → render → review → publish → learning).
- **Source of truth:** Postgres schema `caf_core` + `content_jobs` rows, especially `generation_payload`.
- **Review app** (`apps/review`) is a **client** of Core APIs, not the authority for job state.

Always read first

Resource	Why
<code>docs/CAF_CURRENT_STATE_CONTEXT_PACK.md</code>	Repo-derived current state — start here when context may be stale (BVS, new visual, Why Mimic)
<code>docs/EXTERNAL_CONTEXT_PACK.md</code>	Tiered doc bundle for ChatGPT / other repos / rebuilds
<code>docs/DOMAIN_MODEL.md</code>	ID conventions , <code>task_id</code> joins, lifecycle states (external copy)
<code>.cursor/rules/caf-domain-model.mdc</code>	Same domain rules (Cursor always-on)
<code>docs/ARCHITECTURE.md</code>	Layers, critical files, QC/learning split
<code>src/config.ts</code>	Env flags (Zod); defaults for validation, human review after QC, URLs

Invariants — do not break without explicit user approval

1. `task_id` is the primary execution key across `job_drafts`, `assets`, `editorial_reviews`, `job_state_transitions`, etc., scoped by `project_id`.
2. Prefer joins on `(project_id, task_id)` or `(project_id, run_id)` using **text IDs** as documented in `caf-domain-model`.
3. `content_jobs.generation_payload` is the main JSON contract; coordinate changes with pipeline, Review, and admin consumers. Use `mergeGenerationPayloadQc` (`src/domain/generation-payload-qc.ts`) to write the `qc_result` slice — it is the only sanctioned writer for that subset.

4. **HeyGen retry**: avoid double-submitting renders when `render_state` already holds `video_id` / `session_id`. Use `hasActiveProviderSession` from `src/domain/content-job-render-state.ts` — do not re-implement the check inline.
5. **Learning lookups go through the facade** at `src/services/learning-rule-selection.ts` — `getLearningRulesForPlanning` for planning, `getLearningContextForGeneration` for prompts. Do not import `listActiveAppliedLearningRules` or `compileLearningContexts` directly from new code.
6. **QC risk enforcement today**: `risk_policies` (scoped per `flow_type` via `applies_to_flow_type`) + `brand_constraints.banned_words`. Project `risk_rules` are **not** enforced by `qc-runtime.ts`; **GET** `/v1/projects/:slug/risk-qc-status` is the canonical honesty endpoint.
7. **Review app** `/v1/ contract` is covered by `src/routes/review-contract.test.ts`. If you rename or remove a listed path, update that test in the same change.
8. **Typed payload readers** — prefer the helpers in `src/domain/` over ad-hoc casts:
 - `pickGeneratedOutput` / `hasGeneratedOutput` for `generation_payload.generated_output`
 - `pickRenderState` / `hasActiveProviderSession` / `isMidProviderPhase` for `render_state`
 - `pickStoredQcResult` / `mergeGenerationPayloadQc` for `qc_result`
 - `pickMimicPayload` for `generation_payload.mimic_v1`
 - `parseBvsV1` / `attachBvsToPlannedPayload` for `generation_payload.bvs_v1`
 - `pickMimicCarouselDraftPackage` for `mimic_carousel_package` (TP-grounded carousel flows only)
9. **Structured logs**: when adding new pipeline logging, prefer `logPipelineEvent(level, stage, message, { run_id, task_id, job_id, ... })` from `src/services/pipeline-logger.ts` over `console.*`. Correlation fields are how we trace a single job across stages.
10. **Upstream recommendations** (`src/domain/upstream-recommendations.ts`) are the structured shape the post-approval LLM reviewer emits. Writers must use `parseUpstreamRecommendations` (tolerant) and persist via `insertLlmApprovalReview`; each parsed item is also logged as its own `learning_observation` — do not bypass either when extending the reviewer.
11. **Run context snapshot** (`src/services/run-context-snapshot.ts`) is the canonical record of "what we actually generated with" (prompt versions + brand/strategy slice + learning fingerprints). Persist via `setRunContextSnapshot`. Snapshot failures are logged but must never abort a run.
12. **Mimic carousel package** — `mimic_carousel_package` is for **TP-grounded carousel render flows** (`isTpGroundedCarouselRenderFlow()`): `FLOW_TOP_PERFORMER_MIMIC_CAROUSEL`, `FLOW_VISUAL_FIRST_CAROUSEL`, `FLOW_WHY_MIMIC_CAROUSEL`. Do not conflate with `FLOW_CAROUSEL` / `carousel_package`. Render truth is `mimic_v1` on `generation_payload`. Mimic LLM

prompts must filter signal pack to the job's single planned idea (`llm-creation-pack-budget.ts` , `mimicFlowOnly: true`).

13. **Mimic carousel text overlay** — TP-grounded carousels must **not** bake LLM copy into image models at render. Overlay-only (DocAI/HBS + `reprint-text-overlay`) is enforced in `job-pipeline.ts` ; do not re-enable Flux typography for carousels without explicit approval.
14. **New visual carousel** — `FLOW_VISUAL_FIRST_CAROUSEL` uses `execution_mode: "new_visual"` on `mimic_v1` : idea + BVS driven, **no** top-performer `reference_items` . Prep in `new-visual-carousel-prep.ts` — not TP reference resolve. Do not treat visual-first as competitor frame replication.
15. **Why Mimic carousel** — `FLOW_WHY_MIMIC_CAROUSEL` uses `execution_mode: "why_mimic"` with Slide Intelligence (`slide_intelligence` on `mimic_v1`). Same render engine as other TP-grounded flows; copy/prompts are SIL-driven.
16. **Brand Visual System (BVS)** — versioned `brand_bibles` (`brand_bible_v1`) snapshotted to `generation_payload.bvs_v1` when `candidate_data.use_brand_visual_system` is set (visual-first defaults on). `mimic_v1.bvs_render_plan` drives invented plates for `template_bg` . Use `src/domain/brand-bible.ts` , `bvs-v1.ts` , `bvs-render-plan.ts` — do not ad-hoc read bible JSON from jobs.

Where to change behavior

Goal	Start here
Run planning, scoring, suppression	<code>src/decision_engine/</code> , <code>src/services/run-orchestrator.ts</code>
LLM prompts & creation pack	<code>src/services/llm-generator.ts</code> , <code>src/repositories/flow-engine.ts</code>
QC checks / risk keywords	<code>src/services/qc-runtime.ts</code> , checklist rows, scoped <code>risk_policies</code> via <code>listRiskPoliciesForJob</code>
Job lifecycle & render	<code>src/services/job-pipeline.ts</code>
Top-performer mimic (prep, render, modes)	<code>src/services/mimic-draft-prep.ts</code> , <code>mimic-carousel-render.ts</code> , <code>mimic-image-job.ts</code> , <code>mimic-mode-classifier.ts</code> , <code>src/domain/mimic-payload.ts</code>
New visual carousel (<code>FLOW_VISUAL_FIRST_CAROUSEL</code>)	<code>src/services/new-visual-carousel-prep.ts</code> , <code>new-visual-carousel-execution.ts</code> , <code>new-visual-carousel-flux-prompts.ts</code>
Why Mimic carousel (<code>FLOW_WHY_MIMIC_CAROUSEL</code>)	<code>src/domain/why-mimic-carousel-flow-types.ts</code> , <code>why-mimic-execution.ts</code> , SIL ON <code>mimic_v1</code>
Brand Visual System (BVS)	<code>src/domain/brand-bible.ts</code> , <code>bvs-v1.ts</code> , <code>bvs-render-plan.ts</code> , <code>src/services/bvs-render-overlays.ts</code> , <code>src/repositories/brand-bibles.ts</code>
Mimic text overlay / DocAI / reprint	<code>src/domain/mimic-docai-layer-positions.ts</code> , <code>mimic-post-render-layout-loop.ts</code> , Review <code>reprint-text-overlay</code>
Human review & rework	<code>src/routes/v1.ts</code> , <code>src/services/rework-orchestrator.ts</code>
Publications	<code>src/routes/publications.ts</code>
Learning APIs & compiled guidance	<code>src/routes/learning.ts</code> , facade at <code>src/services/learning-rule-selection.ts</code>

New HTTP routes	Register in <code>src/server.ts</code> (pattern: <code>register*Routes</code>)
-----------------	---

Conventions

- **TypeScript ESM** — imports use `.js` extensions in source (Node ESM).
- **Tests:** Vitest — `npm test`; colocate `*.test.ts` under `src/`.
- **Migrations:** SQL in `migrations/`; applied via `src/cli/migrate.ts` / startup.
- **Avoid** drive-by refactors in files unrelated to the task; match existing style.

Commands (from repo root)

```
npm run dev          # Core API (watch)
npm run migrate      # Apply DB migrations
npm test            # Vitest
npm run process-run -- <run_id|uuid> [--project SLUG]
```

Review app: `cd apps/review && npm run dev` (needs `CAF_CORE_URL`).

Production Review / Admin UI: embedded in the Core Fly image — **not** a separate Vercel deploy. Canonical URL: <https://caf-core.fly.dev/admin/workbench>. After `apps/review/` changes, ship with **fly deploy -a caf-core** from repo root (rebuilds Next.js standalone in `Dockerfile`). Vercel projects (`caf-core-review`, etc.) are optional/legacy; do not assume they are what operators use.

Documentation map

Doc	Audience
<code>docs/CAF_PRODUCT_PITCH.md</code>	Leadership / investors / evaluators
<code>docs/CAF_COMPLETE_PRODUCT_GUIDE.md</code>	Complete product guide — what CAF is and does
<code>docs/CAF_CURRENT_STATE_CONTEXT_PACK.md</code>	Current repo truth — operational map; PDF volumes in <code>docs/export/pdf/11-14</code>
<code>docs/EXTERNAL_CONTEXT_PACK.md</code>	ChatGPT / external repos — what to upload, system prompt
<code>docs/REBUILD_FROM_DOCS.md</code>	Engineers bootstrapping from scratch
<code>docs/DATABASE_SCHEMA.md</code>	Postgres table catalog
<code>docs/DOMAIN_MODEL.md</code>	Entities, IDs, lifecycles
<code>README.md</code>	Quick start, API index, deploy
<code>docs/CAF_CORE_COMPLETE_GUIDE.md</code>	Single-file full project reference
<code>docs/PROJECT_OVERVIEW.md</code>	Stakeholders / onboarding
<code>docs/ARCHITECTURE.md</code>	Engineers

docs/LIFECYCLE.md	Run & job states
docs/TECH_STACK.md	Stack & services
docs/layers/README.md	Per-layer deep dives
docs/QUALITY_CHECKS.md	QC runtime
docs/GENERATION_GUIDANCE.md	Prompt guidance
docs/RISK_RULES.md	Risk policies vs project risk_rules
docs/MIMIC_FLOWS_COMPLETE_GUIDE.md	Top-performer mimic (full; lags new visual / BVS — prefer current-state pack)
docs/MIMIC_IMAGE_FLOWS.md	Mimic image quick ref
docs/MIMIC_TEXT_PLACEMENT_AUTOMATION.md	Future mimic text placement QA (post-render composite loop) — read before automating
docs/CREATIVE_INTELLIGENCE.md	Top-performer ingest upstream of mimic
docs/API_REFERENCE.md	HTTP examples

Optional rules (Cursor)

- `scene-assembly-n8n-legacy.mdc` — when touching scene/n8n-legacy paths
- `mimic-signal-pack-llm-filter.mdc` — mimic LLM creation pack; single-idea `signal_pack` filter
- `mimic-carousel-package.mdc` — `mimic_carousel_package` vs `FLOW_CAROUSEL` / `carousel_package`
- `visual-first-carousel-flow.mdc` — `FLOW_VISUAL_FIRST_CAROUSEL` : **new visual** lane (`execution_mode: "new_visual"` , idea+BVS, no TP references); same mimic **render engine** as manual mimic; Review workbench without original-vs-generated compare
- `mimic-text-placement-automation.mdc` — **before automating mimic text placement**: post-render composite QA loop, HTML overlay invariants, `docai_layer_positions` schema
- `caf-domain-model.mdc` — **alwaysApply**

If the user's request conflicts with **domain invariants**, **ask** or **surface the tradeoff** before renaming IDs or changing status enums.